

ผล Cross-Dataset ครั้งแรก เทรนด้วย CICIDS2017 ทดสอบกับ CSE-CIC-IDS2018

ในวันที่ 9 สิงหาคม 2569

เป้าหมายของรอบนี้คือทำให้ cross-dataset รันได้ก่อน ยังไม่เน้นว่าผลต้องดี เอกสารนี้รายงาน ผลที่ได้ วิธีที่ใช้ และข้อจำกัดที่ยังเหลืออยู่

ผลการทดลองเดิมทั้งหมดไม่ถูกแก้ไข ทุกอย่างที่รันในรอบนี้เขียนออกมาเป็นไฟล์ชุดใหม่

1. สรุปสั้น

cross-dataset รันได้แล้ว และผลออกมาไม่ดี แต่ตัวเลขที่ได้บอกอะไรได้มากกว่าคำว่าไม่ดี

โมเดลที่เทรนจาก 2017 แทนตรวจจับการโจมตีใน 2018 ไม่ได้เลย สิ่งเดียวที่ถ่ายทอดข้ามชุดข้อมูล ได้คือความสามารถในการบอกว่าอะไรคือ traffic ปกติ

ตัวเลขที่ทำให้เห็นภาพชัดที่สุดคือการเทียบกับการเดาแบบไม่ใช้โมเดลเลย ถ้าทายว่าทุกแถวเป็น Benign จะได้ accuracy 0.8307 และ macro-F1 0.1296 ส่วน random forest ที่เทรนมาจาก 2017 ได้ accuracy 0.8310 และ macro-F1 0.1348 ซึ่งแทบไม่ต่างกัน และ mlp กับ logistic regression ได้ accuracy ต่ำกว่า การเดาเสียอีก

เมื่อเทียบกับโมเดลที่เทรนด้วย 2018 เองภายใต้เงื่อนไขที่คุมให้ตรงกันทุกอย่าง ราคาของการข้ามชุดข้อมูลอยู่ระหว่าง 0.44 ถึง 0.65 macro-F1 ขึ้นกับโมเดล

2. ผลนี้ดีหรือแย่ และอ่านอย่างไร

2.1 คำตอบตรง ๆ

แย่มาก ในแง่ประสิทธิภาพ โมเดลที่เทรนจาก 2017 ยังไม่อยู่ในระดับที่น่าไปใช้ตรวจจับการโจมตี บนข้อมูล 2018 ได้จริง

แต่ในแง่ของการทดลอง ผลชุดนี้ใช้ได้ เพราะมันวัดขนาดของปัญหาออกมาเป็นตัวเลขได้ และชี้ว่าปัญหา อยู่ตรงไหน ซึ่งเป็นสิ่งที่ยังไม่เคยมีมาก่อน ผลลบที่อธิบายได้ต่างจากผลลบที่ไม่รู้สาเหตุ

2.2 อย่าอ่านตัวเลขลอย ๆ ต้องมีสองเส้นกำกับเสมอ

macro-F1 เท่ากับ 0.24 ไม่ได้บอกอะไรเลยถ้าไม่รู้ว่าจะอะไรคือพื้นและอะไรคือเพดาน

เส้น	ค่า	ความหมาย
พื้น	0.1296	ทายว่าทุกแถวเป็น Benign โดยไม่ใช้โมเดล
เพดาน	0.66 ถึง 0.80	เทรนด้วย 2018 เอง ทดสอบบนแถวเดียวกัน
ที่ได้จริง	0.13 ถึง 0.24	เทรนด้วย 2017

ค่าที่ได้เกาะอยู่แถวพื้นมากกว่าเพดาน

2.3 กู้คืนได้กี่เปอร์เซ็นต์

วิธีอ่านที่ตรงที่สุดคือถามว่า จากช่องว่างทั้งหมดระหว่างพื้นกับเพดาน โมเดลที่ข้ามชุดข้อมูลมา กู้คืนได้กี่เปอร์เซ็นต์

โมเดล	เพดาน	ที่ใต้	กู้คืนได้
stacking	0.6866	0.2428	20.3%
lightgbm	0.7465	0.2298	16.2%
catboost	0.7954	0.2119	12.4%
xgboost	0.7463	0.1919	10.1%
logistic regression	0.6562	0.1673	7.2%
mlp	0.7216	0.1676	6.4%
random forest	0.7895	0.1348	0.8%

ตัวที่ดีที่สุดกู้คืนได้ราวหนึ่งในห้า ตัวที่แย่ที่สุดแทบไม่กู้คืนอะไรเลย

ถ้าตัวเลขนี้อยู่แถว 70 ถึง 80 เปอร์เซนต์ จะเรียกได้ว่า generalisation พอไปได้ ถ้าอยู่แถว 40 ถึง 50 เปอร์เซนต์ จะเรียกว่าพอมีความหวังแต่ต้องปรับอีกมาก ที่ 0.8 ถึง 20 เปอร์เซนต์คือยังไม่ทำงาน

2.4 ย่ออ่าน accuracy

accuracy ของงานนี้อยู่ที่ 0.72 ถึง 0.85 ซึ่งดูเหมือนใช้ได้ แต่เป็นตัวเลขที่หลอก เพราะข้อมูล ทดสอบเป็น Benign อยู่ 83 เปอร์เซนต์ โมเดลที่ตอบว่า Benign ทุกแถวก็ได้ 0.8307 แล้ว

ในงานที่ข้อมูลเอียงขนาดนี้ accuracy บอกได้แค่สัดส่วนของคลาสใหญ่ ไม่ได้บอกความสามารถของโมเดล ให้ดู macro-F1 กับ F1 รายคลาสแทน

2.5 คลาสไหนอ่านได้ คลาสไหนอ่านไม่ได้

ก่อนตีความ F1 ของคลาสใด ให้ดูจำนวน test flow ก่อนเสมอ

คลาส	test flow	อ่านได้ไหม
Benign	74,769	อ่านได้
DDoS	7,009	อ่านได้
DoS	3,629	อ่านได้
Brute Force	2,112	อ่านได้
Bot	1,587	อ่านได้
Infiltration	898	อ่านได้
Web Attack	7	อ่านไม่ได้

Web Attack มี 7 แถว การเปลี่ยนค่าตอบเพียงแถวเดียวทำให้ F1 ขยับได้มหาศาล ตัวเลขของคลาสนี้จึงอธิบายตัวอย่างที่หยาบมาได้ ไม่ได้อธิบายความสามารถของโมเดล ทุกครั้งที่รายงาน macro-F1 จึงควรรายงานคู่กับค่าที่ตัดคลาสนี้ออกด้วย

2.6 สรุปได้อะไร และยังไม่สรุปไม่ได้อะไร

สรุปได้

1. โมเดลที่เทรนจาก 2017 ตรวจจับการโจมตีใน 2018 ไม่ได้ในทางปฏิบัติ
2. ความรู้ที่ถ่ายทอดข้ามชุดข้อมูลได้คือความรู้เรื่อง traffic ปกติ ไม่ใช่ความรู้เรื่องการโจมตี
3. ขนาดของปัญหาใหญ่กว่าความต่างระหว่างโมเดลราวสิบเท่า การเปลี่ยนโมเดลจึงแก้ไม่ได้

ยังไม่สรุปไม่ได้

1. ยังไม่รู้ว่าที่พังเป็นเพราะ domain shift จริง เพราะคุณภาพ label หรือเพราะ feature ที่บ่งบอกมาจาก dataset ไหน
2. ยังไม่รู้ว่าที่สกลับ 2018 ไป 2017 จะพังเท่ากันหรือไม่
ถ้าไม่เท่ากันจะเป็นหลักฐานว่าคุณภาพของสองชุดข้อมูลต่างกัน
3. ยังไม่รู้ว่าเลขที่ได้เสถียรแค่ไหน เพราะรันด้วย seed เดียว
4. ยังไม่รู้ว่าตัวเลขจะเปลี่ยนไปแค่ไหนถ้าเทรน 2017 ใหม่ด้วย 7 คลาสตั้งแต่ต้น

3. ทำได้อย่างไรทั้งที่ไม่มีข้อมูลดิบ 2017

บันเดิลผลการเทรน `cicids2017_temporal_v1` บรรจุโมเดลที่เทรนเสร็จแล้วครบเจ็ดตัว แต่ละตัวเป็น pipeline ที่มี imputer, scaler และ classifier อยู่ในตัว โดย fit ด้วยสถิติของ 2017 ทั้งหมด จึงไม่จำเป็นต้องมีข้อมูลดิบ 2017 เพื่อทดสอบทิศทาง 2017 ไป 2018

ที่มาของโมเดล โมเดลชุดนี้ไม่ได้เทรนขึ้นใหม่ในรอบนี้ แต่นำมาจากผลการเทรนที่มีอยู่แล้ว จึงยังไม่สามารถยืนยันได้ว่าให้ผลตรงกับตอนที่เทรน รายละเอียดอยู่ในหัวข้อ 4.1

ขั้นตอน

1. โหลดตัวอย่าง 300,000 แถวของ 2018
2. แปลงชื่อคอลัมน์เข้าสู่ schema กลางด้วย `to_canonical` ได้ 77 คอลัมน์
3. แปลงค่าอนันต์เป็นค่าว่าง แล้วปล่อยให้ imputer ของ 2017 เติมด้วย median ของ 2017
4. ให้โมเดล 2017 ทำนาย
5. ยุบหึ่งค่าทำนายและเฉลี่ยเป็น 7 คลาสรวม
6. วัดผลบนชุดทดสอบ 90,011 แถว

ยืนยันสำคัญเรื่อง mapping ไฟล์ `feature_columns.json` ของบันเดิล 2017 ตรงกับ `CANONICAL_FEATURES` ที่สร้างขึ้นทั้ง 77 ตัว เรียงตรงกันทุกตำแหน่ง mapping ที่อนุมานจากลำดับคอลัมน์จึงได้รับการยืนยันจากไฟล์จริงของฝั่ง 2017 แล้ว

4. การตรวจสอบก่อนเชื่อผล

4.1 ไฟล์โมเดล

โมเดลถูกบันทึกด้วย `scikit-learn 1.8.0` แต่เครื่องที่รันใช้ `1.9.0` จึงตรวจโครงสร้างก่อนใช้ ทั้งเจ็ดตัวรายงานว่ารับ input 77 คอลัมน์และมี 9 คลาสตรงตามที่ควรเป็น

ข้อจำกัดที่ต้องระบุดังนี้ คือ ยังไม่ได้ตรวจว่าโมเดลให้ผลตรงกับ `metrics.json` เดิมหรือไม่ เพราะการตรวจนั้นต้องใช้ test set ของ 2017 ซึ่งไม่มีในเครื่อง สิ่งที่ตรวจได้คือโครงสร้าง ไม่ใช่ ค่าตัวเลข

4.2 การจัดการค่าอนันต์

ข้อมูล 2018 มีค่าอนันต์ 2,423 ค่า ซึ่งทำให้ imputer ของ 2017 หยุดทำงานทันที เป็นตัวอย่างของปัญหาที่ต้องทำความสะอาดเองก่อนเชื่อมสองชุดข้อมูลเข้าด้วยกัน

ฝั่ง 2017 ใช้วิธีแปลงค่าอนันต์เป็นค่าว่างแล้วทิ้งแถวนั้น แต่ที่นี้เลือกแปลงเป็นค่าว่างแล้วให้ imputer ของ 2017 เติมค่าแทน เพื่อรักษาชุดทดสอบให้เป็นแถวเดียวกับที่ใช้วัดผลภายในชุด 2018 รวมค่าว่างที่ imputer ต้องเติมทั้งหมด 3,476 ค่า จาก 23,100,000 ค่า คิดเป็น 0.015 เปอร์เซ็นต์

5. ผลลัพธ์ ทดสอบบนชุดทดสอบ 90,011 แถว

เทียบกับการเดาว่าทุกแถวเป็น Benign ซึ่งได้ accuracy 0.8307 และ macro-F1 0.1296

โมเดล	accuracy	macro-F1
stacking	0.8482	0.2428
lightgbm	0.8467	0.2298
catboost	0.8456	0.2119
xgboost	0.8340	0.1919
mlp	0.8150	0.1676
logistic regression	0.7162	0.1673
random forest	0.8310	0.1348
การเดาว่า Benign ทั้งหมด	0.8307	0.1296

โมเดลที่ดีที่สุดเอาชนะการเดาได้ 0.113 บน macro-F1 ส่วนโมเดลที่แย่ที่สุดเอาชนะได้ 0.005

6. ผลรายคลาส

คลาส	จำนวนแถว	ดีที่สุด	โมเดลที่ทำได้
Benign	74,769	0.9265	lightgbm
Brute Force	2,112	0.7359	stacking
DDoS	7,009	0.3805	lightgbm
Bot	1,587	0.2169	catboost
DoS	3,629	0.0761	catboost
Infiltration	898	0.0150	logistic regression
Web Attack	7	0.0011	logistic regression

คลาสโจมตีห้าจากหกคลาสได้ค่าต่ำกว่า 0.4 ทุกโมเดล และสามคลาสได้เกือบศูนย์ทุกโมเดล

ข้อสังเกตที่แปลกและควรตามต่อ คือ Brute Force ของ stacking ได้ 0.7359 ขณะที่โมเดลอื่นได้ ตั้งแต่ 0.0019 ถึง 0.3902 ความต่างขนาดนี้ในคลาสเดียวไม่น่าจะเป็นเรื่องบังเอิญ

7. เทียบกับผลเก่า การสร้างตัวเทียบที่ถูกต้อง

7.1 ทำไมต้องรันตัวเทียบใหม่

การเทียบครั้งแรกไม่ถูกต้อง เพราะฝั่งข้ามชุดข้อมูลกับฝั่งภายในชุดต่างกันสองอย่างพร้อมกัน จึงแยกไม่ออกว่าส่วนต่างมาจากอะไร

	เทรนด้วย	feature	การยุบคลาส
ผลเก่า	2018, 7 คลาส	76 ไม่มี Dst Port	ไม่ต้องยุบ
ข้ามชุดข้อมูล	2017, 9 คลาส	77 มี Dst Port	ยุบตอนทำนาย

จึงรันตัวเทียบใหม่สองแบบ ทั้งคู่ใช้ 77 feature เท่ากับฝั่ง 2017 และทดสอบบน 90,011 แถวเดิม

- C7 เทรนด้วย 7 คลาสโดยตรง ตาม protocol ที่กำหนดไว้
- C15 เทรนด้วย 15 คลาสแล้วยุบเป็น 7 ตอนทำนาย ซึ่งเป็นขั้นตอนเดียวกับที่โมเดล 2017 ผ่าน

C15 คือตัวเทียบที่ถูกต้อง เพราะต่างจากฝั่งข้ามชุดข้อมูลเพียงเรื่องเดียว คือเทรนจาก dataset ไหน

7.2 ตารางเทียบ macro-F1 ทุกคลาส

โมเดล	C7	C15	ข้ามชุด	ส่วนต่าง
lightgbm	0.8149	0.7465	0.2298	ลด 0.5167
xgboost	0.7721	0.7463	0.1919	ลด 0.5544
catboost	0.7957	0.7954	0.2119	ลด 0.5835
random forest	0.7889	0.7895	0.1348	ลด 0.6547
mlp	0.7300	0.7216	0.1676	ลด 0.5540
logistic regression	0.6622	0.6562	0.1673	ลด 0.4889
stacking	0.6868	0.6866	0.2428	ลด 0.4438

คอลัมน์ส่วนต่างคือ ข้ามชุด ลบ C15

7.3 ตารางเทียบ macro-F1 เฉพาะคลาสที่วัดผลได้

นับเฉพาะคลาสที่มี test flow ตั้งแต่ 10 แถวขึ้นไป ซึ่งตัด Web Attack ที่มี 7 แถวออก

โมเดล	C7	C15	ข้ามชุด
lightgbm	0.8296	0.8293	0.2681
xgboost	0.8267	0.8290	0.2238
catboost	0.8284	0.8279	0.2472
random forest	0.8294	0.8302	0.1572
mlp	0.8100	0.8002	0.1955
logistic regression	0.7309	0.7239	0.1950
stacking	0.8012	0.8010	0.2833

ภายในชุด 2018 โมเดลกลุ่มต้นไม่อยู่ที่ 0.83 กันหมด ส่วนพอข้ามชุดข้อมูลตกลงมาเหลือ 0.16 ถึง 0.28 ซึ่งเป็นการตกที่ใหญ่กว่าความต่างระหว่างโมเดลใด ๆ ที่วัดมาทั้งโปรเจกต์หลายเท่า

7.4 การเทรนด้วย 7 คลาสตั้งแต่ต้น ให้ผลต่างเพียงเล็กน้อย

โมเดล	C7 ลบ C15
lightgbm	เพิ่ม 0.0684
xgboost	เพิ่ม 0.0258
mlp	เพิ่ม 0.0084
logistic regression	เพิ่ม 0.0060
catboost	เพิ่ม 0.0003
stacking	เพิ่ม 0.0002
random forest	ลด 0.0006

มีเพียง lightgbm ที่ได้ประโยชน์ชัดเจน อีกหกโมเดลขยับไม่ถึง 0.03 และห้าในนั้นขยับไม่ถึง 0.01

และเมื่อดูเฉพาะคลาสที่วัดผลได้ในหัวข้อ 7.3 ส่วนต่างหายไปเกือบหมดทุกโมเดล รวมทั้ง lightgbm ที่เหลือเพียง 0.0003 แปลว่าส่วนต่างเกือบทั้งหมดมาจากคลาส Web Attack ที่มี test flow เพียง 7 แถว ไม่ใช่จากความสามารถที่วัดได้จริง

ข้อสรุปคือ การเทรนด้วย 7 คลาสตั้งแต่ต้นตาม protocol ไม่ได้เสียหาย และเป็นวิธีที่ตรงไปตรงมา กว่า แต่ยังไม่มียหลักฐานว่ามั่นคงกว่าการยุบที่หลังอย่างมีนัยสำคัญ

8. คำทำนายที่ผิดโดยโครงสร้าง

โมเดล 2017 รู้จัก 9 คลาส ซึ่งสองคลาสคือ Heartbleed และ PortScan ไม่มีอยู่ใน 2018 เลย คำทำนายเหล่านั้นจึงผิดแน่นอนไม่ว่ากรณีใด

โมเดล	จำนวนคำทำนายที่ผิดโดยโครงสร้าง
mlp	3,632
logistic regression	3,002
lightgbm	2,569
xgboost	2,088
stacking	265
catboost	139
random forest	0

จาก 90,011 แถว mlp ทำนายผิดแบบนี้ไป 4 เปอร์เซนต์ ขณะที่ random forest ไม่ทำเลยสักครั้ง

9. ข้อค้นพบ

9.1 สิ่งที่ย้ายทอดข้ามชุดข้อมูลได้คือความรู้เรื่อง traffic ปกติ

Benign ได้ F1 0.87 ถึง 0.93 ทุกโมเดล ขณะที่คลาสโจมตีเกือบทั้งหมดพัง ข้อสรุปนี้ให้ทิศทางได้ เพราะมันบอกว่าถ้าจะทำ generalisation ให้ได้ ควรมองไปทาง anomaly detection ที่เรียนรู้ว่าอะไร คือปกติ มากกว่าจะพยายามให้โมเดลจำแนกชนิดการโจมตีข้ามชุดข้อมูล

9.2 accuracy ที่ดูสูงเป็นภาพลวง

accuracy 0.83 ถึง 0.85 ดูไม่แย่ แต่การเดาว่าทุกแถวเป็น Benign ก็ได้ 0.8307 อยู่แล้ว การรายงาน accuracy ของงาน cross-dataset โดยไม่วางคู่กับ baseline นี้ จะทำให้ผลดูดีเกินความจริงมาก

9.3 ขนาดของปัญหาใหญ่กว่าความต่างระหว่างโมเดลมาก

ที่ผ่านมาการเปรียบเทียบโมเดลอยู่ในระดับความต่าง 0.01 ถึง 0.05 และส่วนใหญ่แยกไม่ออกจากความผันผวนของคลาสหายาก แต่ช่องว่างของการข้ามชุดข้อมูลอยู่ที่ 0.44 ถึง 0.65 ซึ่งใหญ่กว่าสิบเท่า การเลือกโมเดลจึงไม่ใช่ประเด็นหลักของปัญหานี้

10. ข้อจำกัดของผลชุดนี้

10.1 จำนวนข้อมูลที่ใช้เทรนไม่เท่ากัน

	จำนวนแถวที่ใช้เทรน
โมเดล 2017	1,748,579
โมเดล 2018	209,989

ต่างกัน 8.3 เท่า ซึ่งเป็นความไม่สมมาตรที่ต้องระบุ แต่ทิศทางของมันไม่ล้มข้อสรุป ด้วยสองเหตุผล

ข้อแรก ข้อได้เปรียบอยู่ฝั่ง 2017 โมเดลที่เห็นข้อมูลมากกว่าแปลเทากลับทำได้แค่ระดับเดียวกับการ เดา การที่ transfer ฟังจึงไม่ได้เกิดจากข้อมูลไม่พอ และถ้าให้ฝั่ง 2018 ได้ข้อมูลเท่ากันบ้าง ตัวเทียบน่าจะสูงขึ้น ทำให้ช่องว่างกว้างกว่าเดิม ตัวเลขที่รายงานจึงเป็นค่าต่ำสุดของช่องว่าง

ข้อสอง ขนาดของผลจากจำนวนข้อมูลวัดไว้แล้วในโปรเจกต์นี้ การเพิ่มจาก 300k เป็น 500k ทำให้ macro-F1 ขยับราว 0.03 ถึง 0.04 และภายใต้ temporal split ทิศทางยังไม่สอดคล้องกันด้วยซ้ำ เทียบกับช่องว่าง 0.5 แล้วต่างกันเกินสิบเท่า

10.2 ข้อจำกัดอื่น

1. ทดสอบได้ทิศทางเดียว ทิศ 2018 ไป 2017 ต้องใช้ test set ของ 2017 ซึ่งยังไม่มีในเครื่อง
2. โมเดล 2017 นำมาจากผลการเทรนที่มีอยู่เดิม ไม่ได้เทรนใหม่ในรอบนี้ และที่ยืนยันไม่ได้ว่าให้ผลตรงกับตอนเทรน ตรวจได้แค่โครงสร้าง
3. ใช้ seed เดียว ยังไม่ได้ทำซ้ำ 5 รอบตาม protocol
4. ยังไม่ได้แยกสาเหตุว่าส่วนต่างมาจาก domain shift จริง จากคุณภาพ label หรือจาก feature ที่บ่งบอก dataset
5. โมเดล 2017 เทรนด้วย 9 คลาส ส่วนตัวเทียบ C15 เทรนด้วย 15 คลาส จำนวนคลาสต้นทางจึงยังไม่เท่ากัน แม้ทั้งสองฝั่งจะผ่านขั้นตอนการยุบเป็น 7 เหมือนกันแล้ว

11. ขั้นตอนต่อไป

1. หาข้อมูล 2017 มาเพื่อทดสอบทิศกลับ และเพื่อยืนยันความถูกต้องของโมเดลที่นำมาใช้ ไม่จำเป็นต้องเป็นไฟล์ดิบ ขอเพียงมี 77 feature กับคอลัมน์ label ก็เพียงพอ
2. ทำ adversarial validation หา feature ที่บ่งบอกว่า record มาจาก dataset ไหน
3. ตามหาสาเหตุว่าทำไม stacking ถึงทำ Brute Force ได้ 0.7359 ขณะที่ตัวอื่นทำไม่ได้
4. ทดลองแนวทาง anomaly detection ตามข้อสังเกตในหัวข้อ 9.1

12. ไฟล์ผลลัพธ์

- cross_2017_to_2018.csv ผลข้ามชุดข้อมูล
- within_2018_matched.csv ตัวเทียบ C7 และ C15
- cross_2017_to_2018_integrity.json ผลการตรวจโครงสร้างโมเดล
- cross_2017_to_2018.py และ within_2018_matched.py สคริปต์ที่ใช้รัน